

INTRODUCTION TO ASP.NET

Introduction to .NET (Microsoft, 2023)

.NET is a free and open-source developer platform for building websites and applications. It is built on a high-performance runtime utilized by several high-scale apps such as Microsoft Teams, UPS mobile app, Forza Horizon, and Allegiance Consulting.

.NET apps and libraries are built from a project file and a source code or using the .NET CLI (command-line interface) or an Integrated Development Environment (IDE) like Visual Studio.

A simple .NET app sample:

Project File: (XML)

```
<Project SDK="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>Program</OutputType>
    <TargetFramework>net7.0</TargetFramework>
  </PropertyGroup>
</Project>
```

Source Code: (C#)

```
Console.WriteLine("Hello, World!");
```

.NET SDK is one of the .NET binary distributions and is a set of tools, libraries, and runtimes for app development, building, and testing.

.NET CLI

```
dotnet run
Hello, World!
```

The app can also be built and run using .NET CLI.

Another binary distribution of .NET is **.NET Runtimes**. The **Common Language Runtime (CLR)** is considered the foundation all .NET apps are built on. .NET is often called a “*managed code*” runtime as it uses a garbage collector for memory allocation and release and because it enforces type and memory safety.

The supported programming languages .NET include:

- **C#** – a general-purpose object- and component-oriented programming language.
- **F#** – a programming language for writing compact, strong, and performant code. It is data-oriented, where code involves transforming data with functions.
- **Visual Basic** – event-driven programming language that provides a graphical user interface (GUI) by simply dragging and dropping objects and modifying their codes.

Three (3) versions of .NET support different types of apps.

1. **.NET Framework** – a software development framework for building and running applications on Windows for easy desktop and web application engineering. It is the original .NET.
2. **Mono** – provides complete SDK features to develop .NET applications on various platforms.
3. **.NET Core** – a cross-platform and open-source implementation of .NET for the cloud age while remaining compatible with the .NET Framework. It is used for Linux, macOS, and Windows apps.

What is ASP.NET? (Microsoft, 2023)

ASP.NET is a framework developed and marketed by Microsoft to allow programmers to build dynamic webpages that run on Windows, Linux, macOS, and Docker. ASP stands for **Active Server Pages**, and ASP.NET pages have `.aspx` file extensions written in C#.

It extends the .NET platform with more tools and libraries for building web apps. These are the following that ASP.NET adds to the .NET platform:

- **Blazor** – allows the creation of interactive web UIs without writing JavaScript
- **Razor Pages**, a web-page templating syntax to develop webpages by putting the server-side C# logic in HTML.
- **Authentication systems** with libraries, databases, and template pages for handling logins, including multi-factor and external authentication requirements for online accounts.

As ASP.NET extends .NET, the diverse ecosystem of packages and libraries for .NET developers becomes available. Authoring libraries is also possible if shared between applications on the .NET platform.

ASP.NET Frameworks

ASP.NET offers three (3) frameworks for creating web applications, each targeting a different development style.

1. **Web Forms**: Building dynamic websites are possible using a drag-and-drop, event-driven model. A design surface and numerous controls and components allow rapidly-build yet powerful UI-driven sites with data access using a rich library of controls with HTML markup encapsulation.
2. **ASP.NET MVC**: Model View Controller, a design pattern separating user interfaces with business logic. This framework includes features that enable fast, TDD-friendly (test-driven development) for creating applications. This has full control over the HTML markup, the separated code and markup, and the easy-to-write test. It is the best choice for mobile and single-page applications.
3. **ASP.NET Web Pages** –a fast, approachable, and lightweight way to combine server code with HTML for dynamic web content like multimedia and social media embeds. It has the HTML markup and codes together in the same file.

These frameworks are based on the .NET Framework and share core functionalities of .NET and ASP.NET. For instance, the frameworks offer a login security model based on membership where they share the same facilities for managing requests and handling sessions. In addition, the frameworks can coexist in the same web application and are not entirely independent.

Advantages of ASP.NET

These are the following advantages that ASP.NET provides:

- **Web APIs**: ASP.NET Web API is a framework for building HTTP services that can be accessed using browsers and mobile devices.
- **Real-time technologies**: ASP.NET SignalR is a new library that makes developing real-time web functionality easier. SignalR allows bidirectional communication between the server and client browser by choosing the best method based on the capabilities of each.
- **Single-page applications (SPA)**: ASP.NET SPA helps build applications with major client-side interactions using HTML 5, CSS 3, and JavaScript that load an HTML webpage and dynamically updates it as the user interacts with the app.
- **Mobile apps and sites**: ASP.NET can power native mobile apps with a Web API back end. The same applies to mobile websites using responsive design frameworks.

ASP.NET Core Essentials (Microsoft, 2023)

ASP.NET Core is a cross-platform, high-performance, open-source framework that runs on .NET Core to build modern, cloud-enabled, and Internet-connected web apps, services, and mobile backends. It can be developed and run on Windows, macOS, and Linux.

It is considered the new and enhanced version of ASP.NET with better functionality and libraries and a comfortable interface. It is also a repurposed version of ASP.NET 4.x with changes resulting in a leaner and more modular framework.

Content Root

The content root defaults to the project's root directory during development. It includes the following:

- The executable hosting the application (.exe).
- Compiled assemblies of the app (.dll).
- Content files used by the app, such as Configuration files (.xml, .json), Razor files (.razor, .cshtml), and Data files (.db)
- Web root – the path for resource files, including images (.png, .jpg), Stylesheets (.css), and JavaScript (.js), usually in a *wwwroot* folder.

Program.cs

ASP.NET Core apps created with web templates have the application startup code in the Program.cs, where services required by the app are configured.

A sample app startup code using C#:

```
var builder = WebApplication.CreateBuilder(args);

// Adds services to the container.
builder.Services.AddRazorPages();
builder.Services.AddControllersWithViews();

var app = builder.Build();

if (!app.Environment.IsDevelopment()) // Configures the HTTP request pipeline.
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles()
app.UseAuthorization();

app.MapGet("/hi", () => "Hello!");
app.MapDefaultControllerRoute();
app.MapRazorPages();

app.Run();
```

Host

An ASP.NET Core app on startup builds a **host** that encapsulates all of the app's resources, such as the following:

- **Logging** – an API that works with various built-in and party logging providers such as Debug and Azure App Service.
- **Configuration** – performed using one or more configuration providers such as Azure Key Vault, Environment Variable, and Key-per-line.
- **Middleware** – software arranged into an app pipeline to control HTTP requests and responses.
- **Dependency Injection (DI) Services** – makes configured services available throughout the app.

These are the three (3) different hosts capable of running an ASP.NET Core app:

1. ASP.NET Core **WebApplication**, or Minimal Host – the most used in all ASP.NET templates.
2. **.NET Generic Host** integrated with ASP.NET Core's **ConfigureWebHostDefaults**
3. ASP.NET Core **WebHost** – available only for backward compatibility.

In comparison, WebApplication behaves similarly to the .NET Generic Host and exposes several of the same interfaces but only needs fewer callbacks to configure.

```
var builder =  
WebApplication.CreateBuilder(args);  
  
// Add services to the container.  
builder.Services.AddRazorPages();  
builder.Services.AddControllersWithViews();  
  
var app = builder.Build();
```

A sample initiating a WebApplication host:

Benefits of ASP.NET Core

These are the following benefits that ASP.NET Core provides:

- Cloud-ready
- Community-focused
- Crafted for testability
- Tooling that simplifies modern web development

Handling Errors

ASP.NET Core has built-in features for handling errors that include the following:

- **Developer Exception Page** – provides complete information about unhandled request exceptions. It is enabled by default when both ASP.NET Core apps run in the Development environment, one of the execution environments available in ASP.NET Core.
- **Custom Error Page** – UseExceptionHandler is called to configure a custom error handling page for the Production environment, another execution environment in ASP.NET Core.
- **Status Code Page** – UseStatusCodePages is used in the Program.cs as an ASP.NET Core app that does not provide a status code page for HTTP error status codes such as **404 – Not Found**.
- **Startup Exception Handling**: Only the hosting layer can handle exceptions during app startup to capture startup and detailed errors.

References:

Birer, K. (2022). *ASP.NET core for jobseekers*. BPB Publications.

Microsoft (2023). *ASP.NET core fundamentals overview*. [Web Article]. Retrieved on May 5, 2023, from <https://learn.microsoft.com/en-us/aspnet/core/fundamentals/>

Microsoft (2023). *ASP.NET overview*. [Web Article]. Retrieved on April 23, 2023, from <https://learn.microsoft.com/en-us/aspnet/overview>

Microsoft (2023). *What is .NET? Introduction and overview*. [Web Article]. Retrieved on May 4, 2023, from <https://learn.microsoft.com/en-us/aspnet/overview>

Shaikh. K. (2020). *Azure.NET for core developers*. BPB Publications.